EN3160 Image Processing and Machine Vision

Assignment 3 – Neural Networks

Name: S.A.D. Priyadarshana

Index: 230502X

GitHub: denethp/CV-Assignment-03

Question 1

[ ]:
```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

%matplotlib inline

_ = torch.manual_seed(0)  # fixed seed so my runs are reproducible
```

[2]:
```
# CIFAR-10, normalized to [-1, 1]
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 50

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')
```

[3]:
```
def accuracy(loader, forward):
    # Percentage of correctly classified images in a loader
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in loader:
            x = inputs.view(inputs.shape[0], -1)
            predicted = torch.argmax(forward(x), dim=1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total
```

Baseline: single dense layer with MSE loss

Before making the required changes, I first trained the given network as it is, so I have a reference to compare against. It maps the flattened image ($3 \times 32 \times 32 = 3072$ values) straight to the 10 class scores with a single weight matrix and is trained with a mean-squared-error loss.

[4]:
```
Din = 3 * 32 * 32  # flattened CIFAR-10 image
K = 10             # number of classes
std = 1e-5

w = torch.randn(Din, K) * std
b = torch.zeros(K)

iterations = 20
lr = 2e-6
lr_decay = 0.9
reg = 0
loss_history = []

for t in range(iterations):
    running_loss = 0.0
    for inputs, labels in trainloader:
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()

        # Forward pass
        y_pred = x_train.mm(w) + b

        # MSE loss with L2 regularization
        loss = (1 / Ntr) * torch.sum((y_pred - y_train_onehot) ** 2) + reg * torch.sum(w ** 2)
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        dy_pred = (2.0 / Ntr) * (y_pred - y_train_onehot)
        dw = x_train.t().mm(dy_pred) + reg * w
        db = dy_pred.sum(dim=0)

        w -= lr * dw
        b -= lr * db

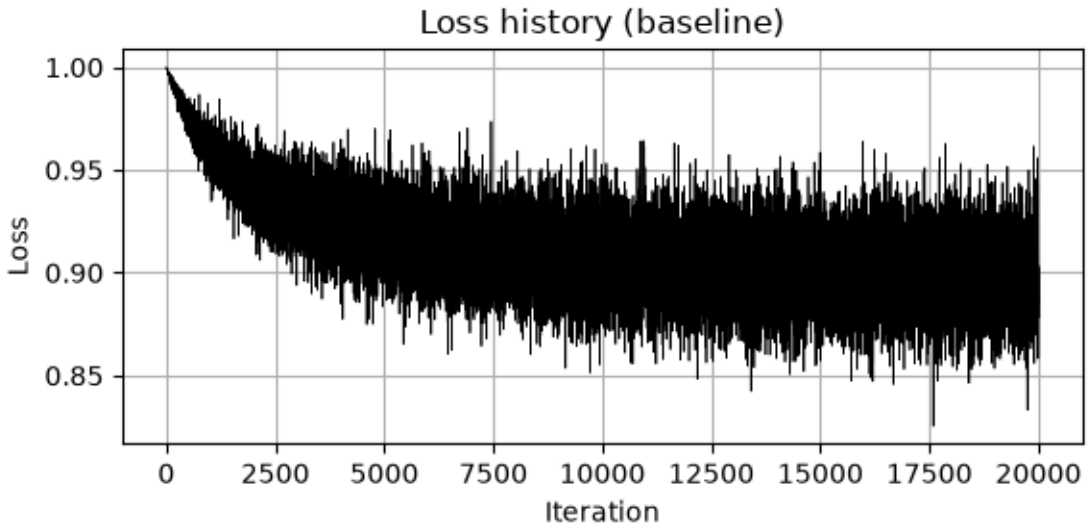
    print(f'Epoch {t + 1}/{iterations}, Loss: {running_loss / len(trainloader):.4f}')
    lr *= lr_decay
```

```
Epoch 1/20, Loss: 0.9769
Epoch 2/20, Loss: 0.9498
Epoch 3/20, Loss: 0.9361
Epoch 4/20, Loss: 0.9275
Epoch 5/20, Loss: 0.9216
Epoch 6/20, Loss: 0.9172
Epoch 7/20, Loss: 0.9138
Epoch 8/20, Loss: 0.9111
Epoch 9/20, Loss: 0.9088
Epoch 10/20, Loss: 0.9070
Epoch 11/20, Loss: 0.9054
Epoch 12/20, Loss: 0.9041
Epoch 13/20, Loss: 0.9030
Epoch 14/20, Loss: 0.9020
Epoch 15/20, Loss: 0.9011
Epoch 16/20, Loss: 0.9004
Epoch 17/20, Loss: 0.8997
Epoch 18/20, Loss: 0.8991
Epoch 19/20, Loss: 0.8986
Epoch 20/20, Loss: 0.8982
```

[5]:
```
baseline_train_acc = accuracy(trainloader, lambda x: x.mm(w) + b)
baseline_test_acc = accuracy(testloader, lambda x: x.mm(w) + b)
print(f'Training accuracy: {baseline_train_acc:.2f}%')
print(f'Test accuracy: {baseline_test_acc:.2f}%')

plt.figure(figsize=(6, 3))
plt.plot(loss_history, color='k', linewidth=0.5)
plt.title('Loss history (baseline)')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Training accuracy: 32.22%
Test accuracy: 32.44%
```



(a) Adding a middle layer with 100 nodes and sigmoid activation

I added a hidden layer of $H = 100$ nodes between the input and the output, so the network becomes

$$\mathbf{h} = \sigma(\mathbf{x}W_1 + \mathbf{b}_1), \qquad \hat{\mathbf{y}} = \mathbf{h}W_2 + \mathbf{b}_2,$$

with $W_1 \in \mathbb{R}^{3072\times100}$ and $W_2 \in \mathbb{R}^{100\times10}$. I still compute the forward pass and the gradients by hand. Going backwards from the output gradient $\partial L/\partial \hat{\mathbf{y}}$:

$$\frac{\partial L}{\partial W_2} = \mathbf{h}^T \frac{\partial L}{\partial \hat{\mathbf{y}}}, \qquad \frac{\partial L}{\partial \mathbf{z}_1} = \left(\frac{\partial L}{\partial \hat{\mathbf{y}}} W_2^T\right) \odot \mathbf{h} \odot (1 - \mathbf{h}), \qquad \frac{\partial L}{\partial W_1} = \mathbf{x}^T \frac{\partial L}{\partial \mathbf{z}_1},$$

where $\mathbf{h} \odot (1 - \mathbf{h})$ is the derivative of the sigmoid. I verified these gradients against numerical (finite-difference) gradients before training.

The tiny initialization (10^{-5}) of the single-layer network does not work with a hidden layer, because every hidden node would output $\sigma(0) = 0.5$ and the signal would barely reach the output. So I initialized each weight matrix with standard deviation $1/\sqrt{\text{fan-in}}$. To speed up training, I loaded the whole dataset onto the GPU once and drew the mini-batches from it directly.

[ ]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

def to_device_tensors(dataset):
    # Same preprocessing as the transform above: [0, 1] -> [-1, 1], then flatten (C, H, W)
    x = torch.tensor(dataset.data, dtype=torch.float32).permute(0, 3, 1, 2) / 255.0
    x = ((x - 0.5) / 0.5).reshape(len(dataset), -1)
    y = torch.tensor(dataset.targets)
    return x.to(device), y.to(device)

X_train, Y_train = to_device_tensors(trainset)
X_test, Y_test = to_device_tensors(testset)
print('Train:', tuple(X_train.shape), ' Test:', tuple(X_test.shape))
```

[ ]:
```
Din = 3 * 32 * 32
H = 100  # hidden nodes
K = 10

def init_params(seed=0):
    g = torch.Generator(device=device).manual_seed(seed)
    w1 = torch.randn(Din, H, device=device, generator=g) / Din ** 0.5
    b1 = torch.zeros(H, device=device)
    w2 = torch.randn(H, K, device=device, generator=g) / H ** 0.5
    b2 = torch.zeros(K, device=device)
    return [w1, b1, w2, b2]

def forward(x, params):
    w1, b1, w2, b2 = params
    h = torch.sigmoid(x.mm(w1) + b1)  # hidden layer
    y_pred = h.mm(w2) + b2            # output scores
    return y_pred, h

def evaluate(X, Y, params):
    with torch.no_grad():
        predicted = torch.argmax(forward(X, params)[0], dim=1)
    return 100 * (predicted == Y).float().mean().item()

def train(loss_fn, epochs=10, lr=0.05, lr_decay=0.9, reg=0.0, seed=0):
    # Mini-batch gradient descent with manually computed gradients
    params = init_params(seed)
    w1, b1, w2, b2 = params
    g = torch.Generator(device=device).manual_seed(seed)
    N = X_train.shape[0]
    loss_history, epoch_loss = [], []

    for epoch in range(epochs):
        perm = torch.randperm(N, device=device, generator=g)
        running_loss = 0.0
        for i in range(0, N, batch_size):
            idx = perm[i:i + batch_size]
            x = X_train[idx]
            y_onehot = nn.functional.one_hot(Y_train[idx], K).float()
```

```
            # Forward pass
            y_pred, h = forward(x, params)
            loss, dy_pred = loss_fn(y_pred, y_onehot)
            loss = loss + reg * (torch.sum(w1 ** 2) + torch.sum(w2 ** 2))

            # Backpropagation
            dw2 = h.t().mm(dy_pred) + 2 * reg * w2
            db2 = dy_pred.sum(dim=0)
            dz1 = dy_pred.mm(w2.t()) * h * (1 - h)  # through the sigmoid
            dw1 = x.t().mm(dz1) + 2 * reg * w1
            db1 = dz1.sum(dim=0)

            # Parameter update
            w1 -= lr * dw1
            b1 -= lr * db1
            w2 -= lr * dw2
            b2 -= lr * db2

            loss_history.append(loss.item())
            running_loss += loss.item()

        epoch_loss.append(running_loss / (N // batch_size))
        print(f'Epoch {epoch + 1}/{epochs}, Loss: {epoch_loss[-1]:.4f}')
        lr *= lr_decay

    return params, loss_history, epoch_loss
```

To see the effect of the hidden layer on its own, I first trained it with the same MSE loss as the baseline, for 10 epochs.

[ ]:
```
def mse_loss(y_pred, y_onehot):
    n = y_pred.shape[0]
    loss = torch.sum((y_pred - y_onehot) ** 2) / n
    dy_pred = 2.0 * (y_pred - y_onehot) / n
    return loss, dy_pred

params_mse, loss_history_mse, epoch_loss_mse = train(mse_loss, epochs=10, lr=0.05)

mse_train_acc = evaluate(X_train, Y_train, params_mse)
mse_test_acc = evaluate(X_test, Y_test, params_mse)
print(f'Training accuracy: {mse_train_acc:.2f}%')
print(f'Test accuracy: {mse_test_acc:.2f}%')
```